

① 1. let $f_1, \dots, f_m: C \rightarrow \mathbb{R}$ be convex functions
 $\& \lambda_1, \dots, \lambda_m \in \mathbb{R}_+$

$$\text{let } g(u) = \sum_{i=1}^m \lambda_i f_i(u) \quad u, v \in C, \alpha \in [0, 1]$$

$$\begin{aligned} g(\alpha u + (1-\alpha)v) &= \sum_{i=1}^m \lambda_i f_i(\alpha u + (1-\alpha)v) \\ &\leq \sum_{i=1}^m \lambda_i (\alpha f_i(u) + (1-\alpha)f_i(v)) = \alpha \cdot \sum_{i=1}^m \lambda_i f_i(u) + (1-\alpha) \cdot \sum_{i=1}^m \lambda_i f_i(v) \\ &\quad \text{each } f_i \text{ is convex} \\ &= \alpha g(u) + (1-\alpha) \cdot g(v) \end{aligned}$$

2. take $f(x) = x^2$ $g(x) = x^2 - 1$ $f'' = g'' = 1 \geq 0$

$$h = f \circ g = (x^2 - 1)^2 = x^4 - 2x^2 + 1$$

$$h'' = 12x^2 - 4 \quad \text{note that } h''(0) = -4$$

$\Rightarrow h$ is not convex

1.2

$$\textcircled{3} \quad x \in \mathbb{R}^d \quad y \in \{\pm 1\}$$

$$f(w, b) = \max_{x, y} \ell(w, b) = \max(0, 1 - y(x^T w + b))$$

$$\text{let } \alpha \in [0, 1] \quad (w, b), (w', b') \in \mathbb{R}^d \times \mathbb{R}$$

$$\text{define } h(w, b) = 0, \quad g(w, b) = 1 - y(x^T w + b)$$

$$\text{note } f(w, b) = \max\{h(w, b), g(w, b)\}$$

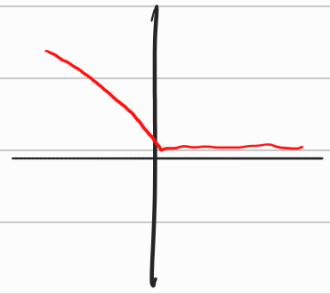
We know from calculus that the max of 2 convex functions is a convex function

h is convex as a constant function

We will show g is convex

Note that $g(w, b) = (-yx)^T w + b + 1$ is affine (linear + const) therefore $\nabla^2 g(w, b) = 0 \succeq 0$
So the Hessian is PSD $\Rightarrow g$ is convex

④ the hinge loss
can be split into 3 regions



• When $y(x^T w + b) > 1$

then $l_{x,y}^{\text{hinge}} = 0$ so $\partial l_{x,y}^{\text{hinge}} = \{\nabla l_{x,y}^{\text{hinge}}\} = \{0\}$

• When $y(x^T w + b) < 1$ so $l_{x,y}^{\text{hinge}} = 1 - y(x^T w + b)$

& differentiable so $\partial l_{x,y}^{\text{hinge}} = \{\nabla l_{x,y}^{\text{hinge}}\}$

$$\nabla l_{x,y}^{\text{hinge}} = \begin{pmatrix} \frac{\partial l}{\partial w} \\ \frac{\partial l}{\partial b} \end{pmatrix} = \begin{pmatrix} -yx^T \\ -y \end{pmatrix}$$

• When $y(x^T w + b) = 1$ we can take any vector under the graph & we can choose one of the gradients we found on the first two cases. Since they are under the graph, for example 0 will always be under the graph of l^{hinge}

⑤ let $f(x) = \sum_{k=1}^m f_k(x)$ by the definition of the subgradient for each convex f_k and any $V \in \mathbb{R}^d$ the following holds:

$$f_k(V) \geq f_k(x) + g_k^T (V - x)$$

$$\Rightarrow \sum_{k=1}^m f_k(V) \geq \sum_{k=1}^m (f_k(x) + g_k^T (V - x))$$

$$\sum f_k(V) \geq \sum f_k(x) + \sum g_k^T (V - x)$$

$$\sum f_k(v) \geq \sum f_k(x) + \left(\sum g_k \right)^T (v-x)$$

$$f(v) \geq f(x) + \left(\sum g_k \right)^T (v-x)$$

this is the definition of the subgradient

$$(6) \quad f(w, b) = \frac{1}{m} \sum_{i=1}^m \lim_{\lambda \rightarrow 0} (v_i, b) + \frac{\lambda}{2} \|w\|^2$$

we can use the linearity property from Q5

$-\frac{\lambda}{2} \|w\|^2$ is convex & smooth so the subgradient is the gradient

$$\nabla_w \frac{\lambda}{2} \|w\|^2 = \lambda w$$

$$\nabla_b \frac{\lambda}{2} \|w\|^2 = 0$$

for each sample i define the indicator

$$I_i = \begin{cases} 1 & \text{if } y_i (v_i^T w + b) < 1 \\ 0 & \text{else} \end{cases}$$

from Q4 the subgradients are $-y_i v_i I_i$ and $-y_i I_i$ for b

$$\Rightarrow \begin{aligned} g_w &= \lambda w - \frac{1}{m} \sum_{i=1}^m (I_i \cdot y_i \cdot x_i) \\ g_b &= -\frac{1}{m} \sum_{i=1}^m (I_i \cdot y_i) \end{aligned} \Rightarrow \begin{pmatrix} g_w \\ g_b \end{pmatrix}$$

$$7 \quad \textcircled{a} \quad \frac{\partial L}{\partial \hat{y}} = y^1 - y \quad \text{since} \quad \hat{y} = w^T h + c$$

$$\frac{\partial L}{\partial h} = (y^1 - y)w, \quad \frac{\partial L}{\partial c} = y^1 - y$$

$$b) \quad \frac{\partial y^1}{\partial w} = \frac{\partial}{\partial h} (w^T h + c) = w$$

$$1. \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial b}$$

$$\frac{\partial h}{\partial b} = \sigma'(wx + b) = \sigma'(wx + b) \cdot 1$$

$$\frac{\partial L}{\partial b} = ((y^1 - y)w) \cdot \sigma'(wx + b)$$

$$2. \quad \frac{\partial L}{\partial w} = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial w}$$

$$\frac{\partial h}{\partial w} = \sigma'(wx + b) \cdot x$$

c) Parts requiring strings

x -input vector used in $w^T x + b$, L (used in $\frac{\partial L}{\partial h}$), y^1

$$8 \quad \textcircled{a} \quad L_i(z) = -\log(z_i)$$

$$\underline{j=i}: \quad \frac{\partial L_i}{\partial z_i} = \frac{\partial}{\partial z_i} (-\log(z_i)) = -\frac{1}{z_i}$$

$$\underline{j \neq i}: \quad \frac{\partial L_i}{\partial z_j} = 0$$

$$\Rightarrow \nabla_z (L_i) = [0, \dots, -\frac{1}{z_i}, \dots, 0] = -\frac{1}{z_i} e_i^T$$

$$\textcircled{b} \quad J_z(l_i) = \frac{1}{z_i} e_i^T, \quad J_x(s) = \text{diag}(S(x)) - S(x)S(x)^T$$

$$J_x(l_i \circ s) = J_z(l_i) \cdot J_x(s) \Rightarrow \left(\frac{1}{z_i} e_i^T \right) (\text{diag}(z) - zz^T)$$

$$\Rightarrow \underbrace{\left(\frac{1}{z_i} e_i^T \text{diag}(z) \right)}_I - \underbrace{\left(\frac{1}{z_i} e_i^T \right)}_{\overline{A}} zz^T$$

$$\textcircled{I} \quad \frac{1}{z_i} e_i^T \text{diag}(z) = z_i e_i^T \cdot \left(\frac{1}{z_i} \right) = e_i^T$$

$$\textcircled{II} \quad \frac{1}{z_i} e_i^T zz^T = \frac{1}{z_i}$$

$$J_x(l_i \circ s) = z^T - e^T$$

Introduction to Machine Learning (67577) — Exercise 3

ID: 322066952

Theoretical Part (Questions 1–8): the full answers are provided as the handwritten pages preceding this section of the document.

Practical Part: the answers and all required plots follow below.

Practical Part — 2.1 Gradient Descent

Question 1 – Descent path at $\eta=0.01$: L1 vs L2

L2 (squared L2 norm, $f(w)=||w||^2$):

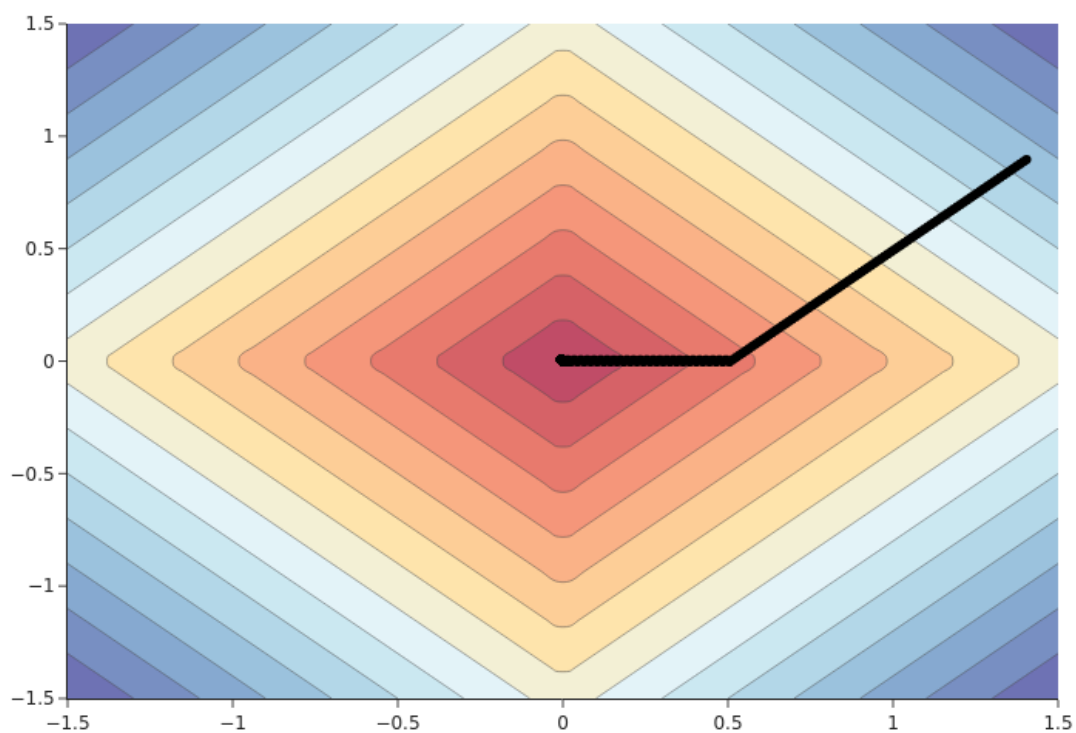
The gradient is $2w$, which is proportional to the distance from the origin. As the algorithm approaches the origin, the gradient magnitude shrinks, producing a smooth, curved path that gradually decelerates. The path spirals inward and converges to the minimum ($w=0$) smoothly.

L1 (L1 norm, $f(w)=||w||_1$):

The gradient is $\text{sign}(w)$, which has constant magnitude (1 in each dimension) regardless of how close w is to the origin. This means the step size in each direction is always the same (η). The path is therefore a piecewise linear "staircase" — diagonal while both components are nonzero, then axis-aligned once a component crosses zero. Near zero, the algorithm may overshoot and oscillate rather than converging precisely, since the gradient does not shrink.

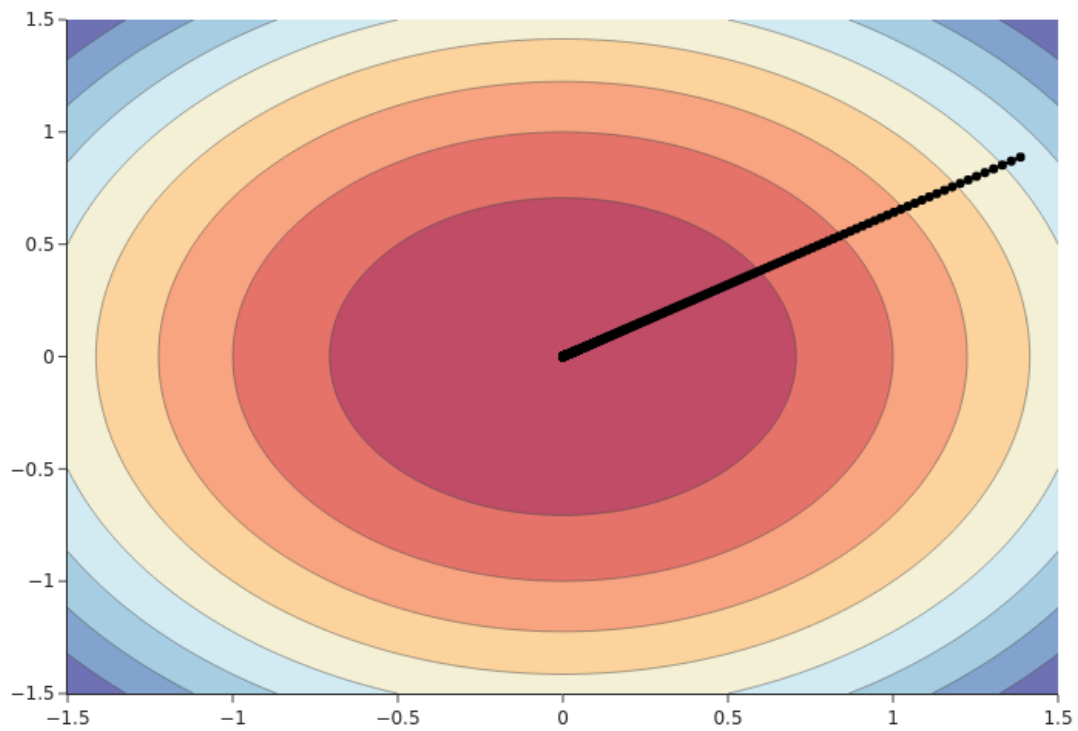
Key difference: L2 has a self-dampening gradient (pulls harder when far, softer when close), enabling smooth convergence. L1's constant-magnitude gradient can cause overshooting near zero and prevents exact convergence for any fixed learning rate larger than zero.

GD Descent Path L1 $\eta=0.01$



L1 descent path, fixed LR $\eta = 0.01$

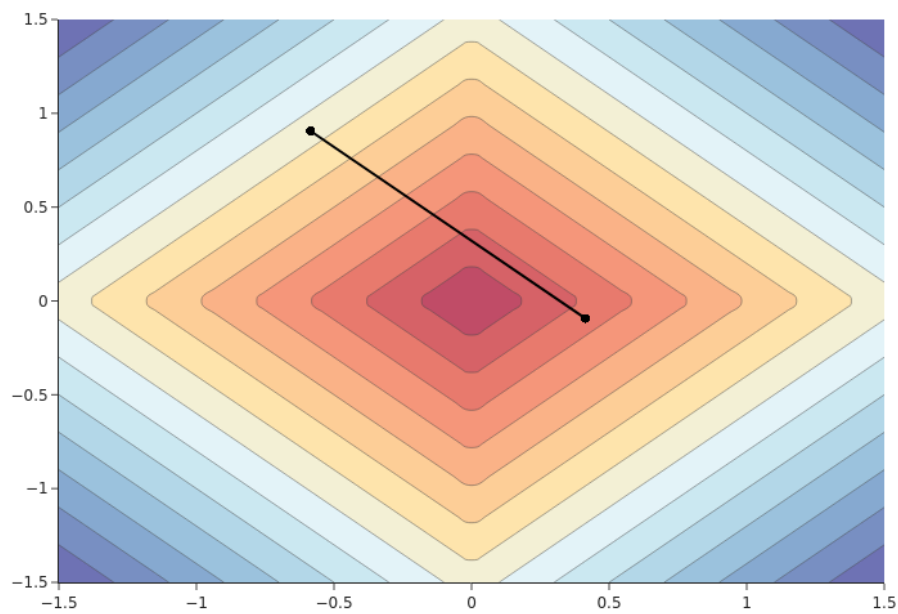
GD Descent Path L2 eta=0.01



L2 descent path, fixed LR $\eta = 0.01$

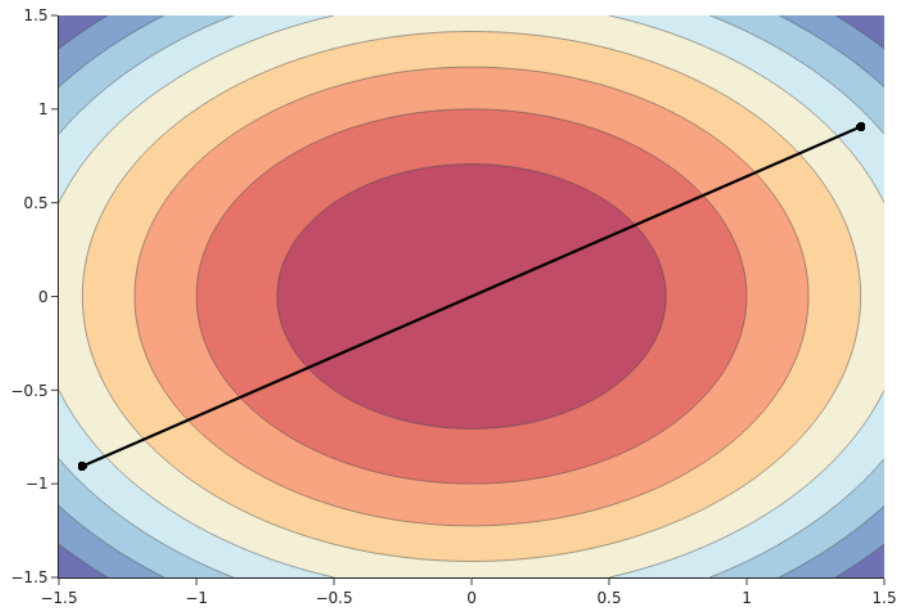
Descent paths for the remaining fixed learning rates (required: “plot the descent path for each setting”):

GD Descent Path L1 eta=1



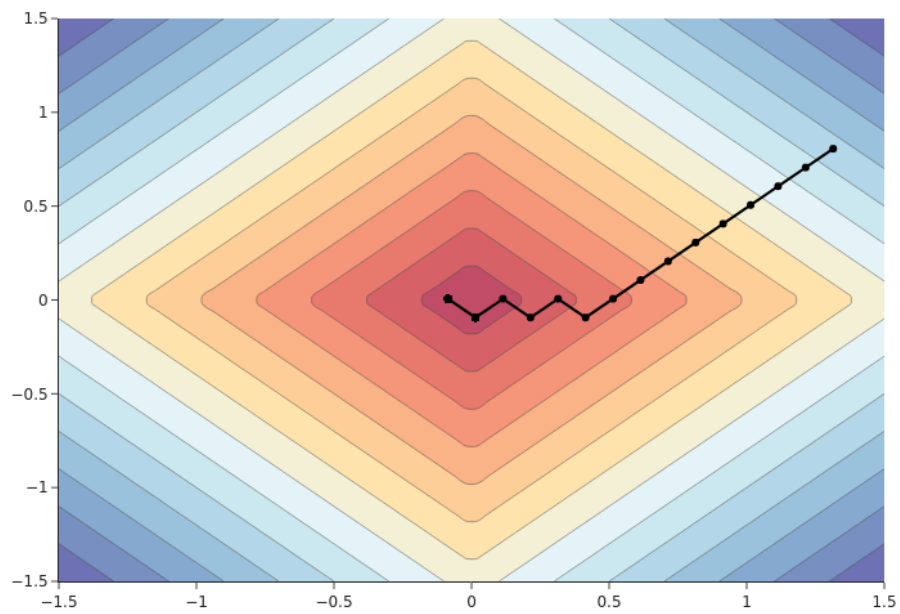
L1 descent path, $\eta = 1$

GD Descent Path L2 eta=1



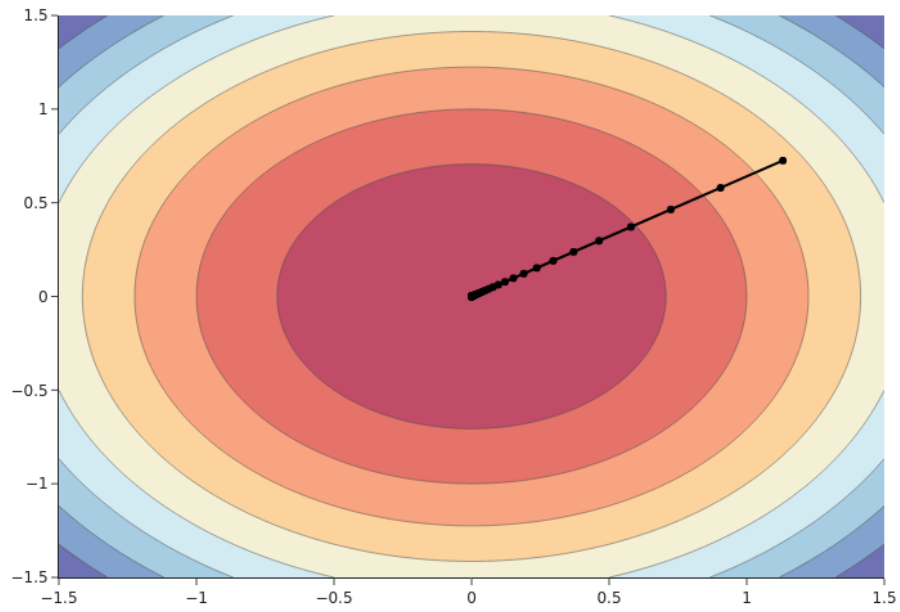
L2 descent path, $\eta = 1$

GD Descent Path L1 eta=0.1



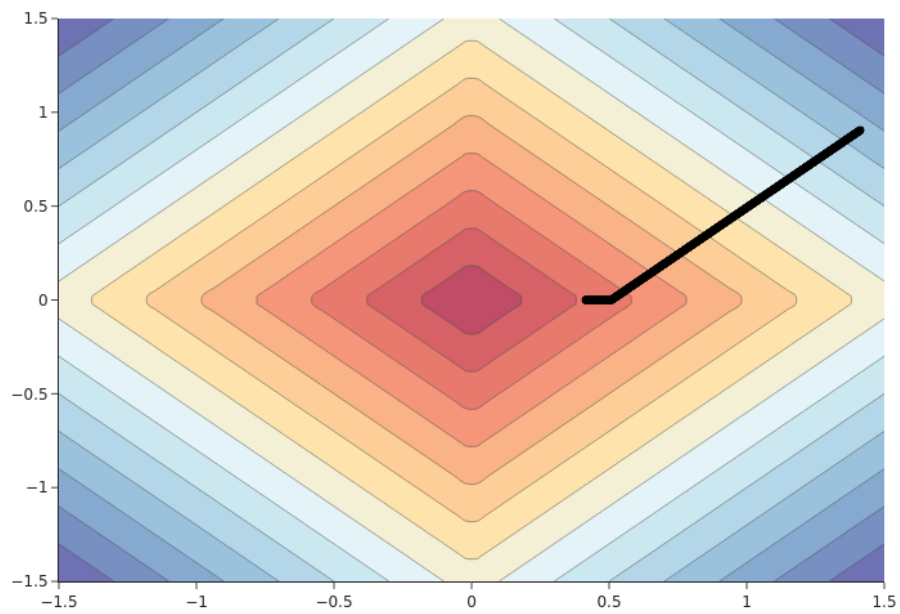
L1 descent path, $\eta = 0.1$

GD Descent Path L2 eta=0.1

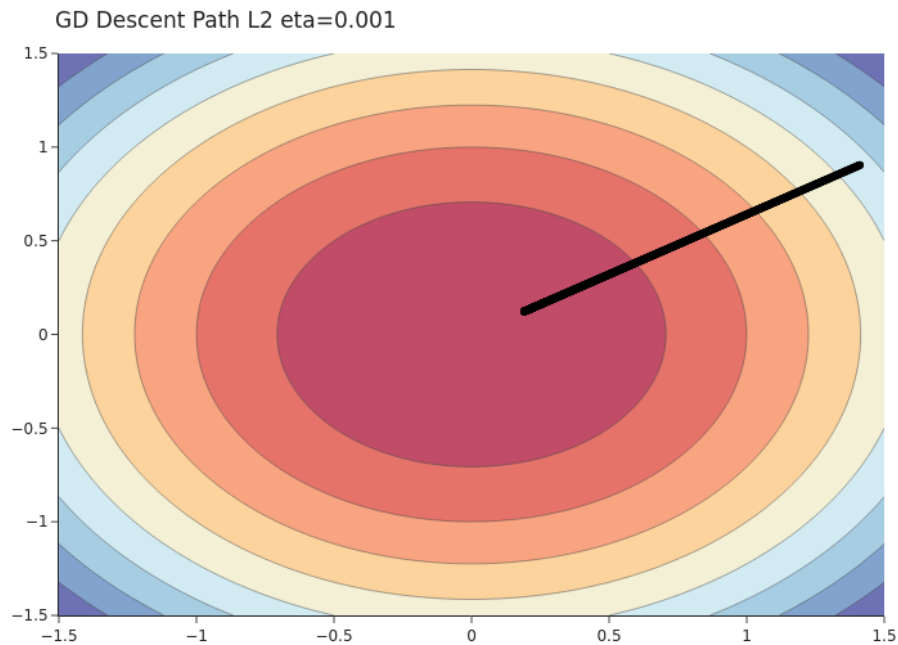


L2 descent path, $\eta = 0.1$

GD Descent Path L1 eta=0.001



L1 descent path, $\eta = 0.001$



Question 2 – Two phenomena in L1 descent path under GD with fixed learning rate

1. Piecewise linear (staircase) trajectory:

Because the L1 gradient is $\text{sign}(w)$ — constant ± 1 per component — each GD update moves w by exactly η in each coordinate direction. The path forms straight diagonal segments while both components are on the same side of zero, then switches to axis-aligned segments once one component reaches (or crosses) zero. The resulting path is a "staircase" rather than a smooth curve.

2. Chattering / oscillation around the minimum:

Once a coordinate is close to zero, the constant-step update can cause it to overshoot and flip sign repeatedly. Since $\text{sign}(w_i)$ flips when w_i crosses zero, the gradient direction alternates each iteration, trapping the iterates in a back-and-forth oscillation of amplitude $\sim \eta$ around zero. This prevents the algorithm from exactly reaching the minimum at $w=0$ for any positive fixed learning rate.

Question 3 – Convergence rate ($\|w\|$ vs iteration) for all learning rates

L2 module:

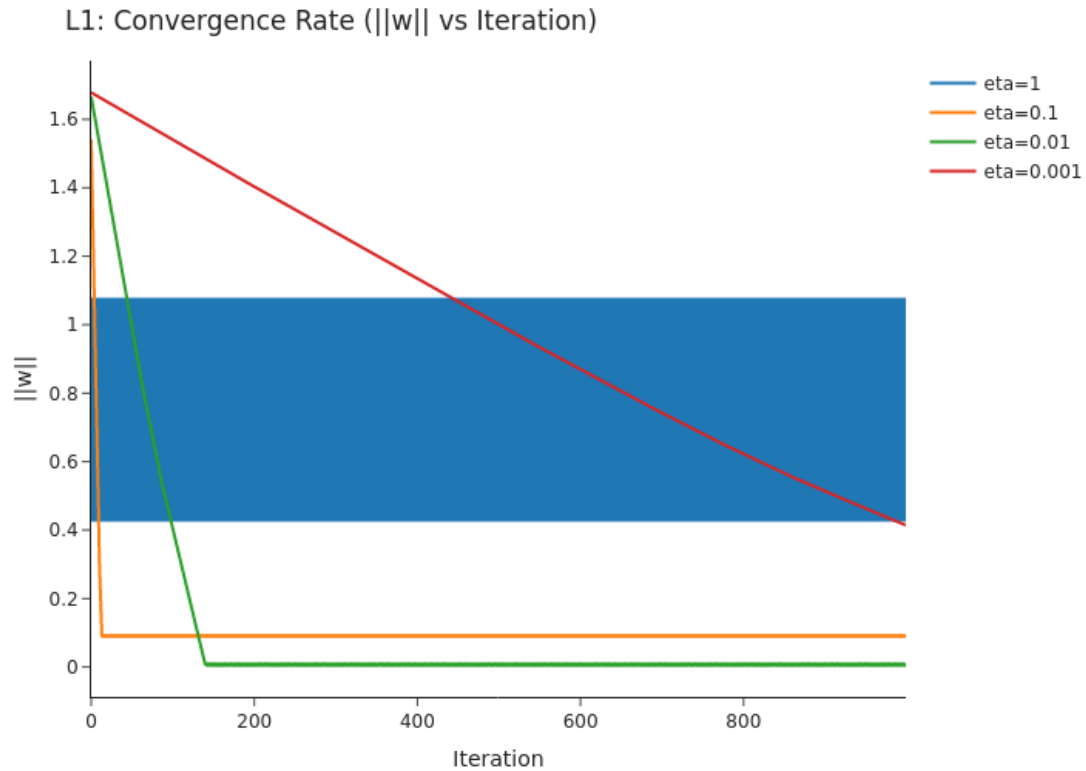
- $\eta=1$: the norm stays exactly constant. The L2 gradient is $2w$, so the update is $w_{\text{new}} = w - 1 * 2w = -w$. Each iteration flips the sign of w without changing its magnitude, so the weights oscillate between $+w_0$ and $-w_0$ forever. The contraction factor $|1 - 2\eta|$ equals exactly 1, giving neither convergence nor divergence — just perpetual oscillation.
- $\eta=0.1$ and $\eta=0.01$: both converge to 0. $\eta=0.1$ converges faster (fewer iterations needed) because it takes larger, yet still stable, steps. Each iteration contracts the norm by the factor $|1 - 2\eta|$: for $\eta=0.1$ the factor is 0.8, for $\eta=0.01$ it is 0.98 — so $\eta=0.1$ is geometrically faster.
- $\eta=0.001$: converges slowly (contraction factor 0.998), reaching a relatively large residual after 1000 iterations.

L1 module:

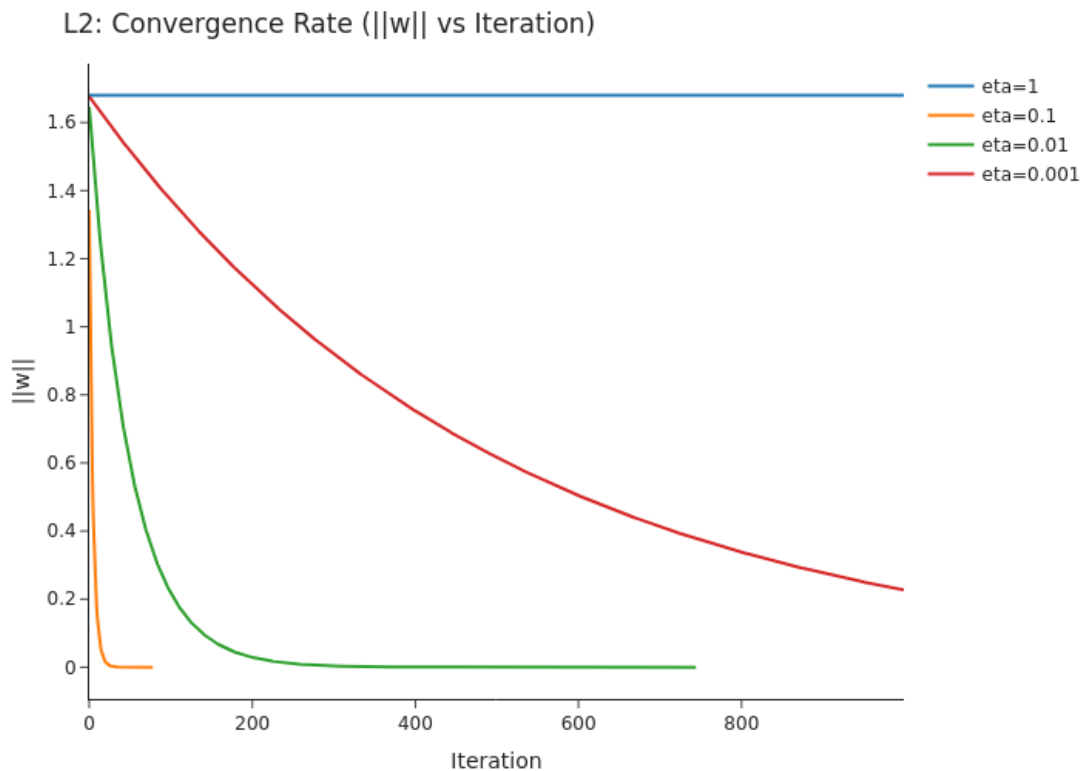
- $\eta=0.001$: decreases very slowly; 1000 steps of size 0.001 move each coordinate by at most 1 unit, leaving $\|w\|$ still large.

- $\eta=0.01$: steadily decreases, reaching a small value near zero.
- $\eta=0.1$: decreases quickly but then oscillates around a small residual.
- $\eta=1$: oscillates with large amplitude — each step of size 1 overshoots zero repeatedly, preventing convergence.

General principle: a learning rate that is too large causes oscillation or divergence; one that is too small converges but very slowly. There is an optimal range that balances speed and stability. the lower the learning rate, the closer we oscillate around the minimum.



L1 convergence (norm vs. iteration), fixed learning rates



L2 convergence (norm vs. iteration), fixed learning rates

Question 4 – Lowest loss achieved per module and explanation

L2 module:

Best $\eta=0.1$ and $\eta=0.01$ both achieve loss ≈ 0 (essentially machine-precision zero).

Because $f(w) = \|w\|^2$ is smooth and strongly convex with a unique minimum at $w=0$, GD with an appropriate learning rate converges geometrically to the exact minimum. The gradient vanishes as $w \rightarrow 0$, so the updates become arbitrarily small and the algorithm can reach $w=0$ to numerical precision.

L1 module:

Best achievable (in 1000 iterations) was $\eta=0.01$ with loss ≈ 0.008 .

Unlike L2, the L1 norm is non-smooth: its gradient is $\text{sign}(w)$, which has constant magnitude regardless of proximity to the minimum. This means:

- (a) The step size never self-dampens near the origin.
- (b) For any fixed $\eta > 0$, the algorithm oscillates with amplitude $\sim \eta$ around zero once it arrives nearby, preventing exact convergence to 0.

Therefore, the lowest achievable loss with a fixed learning rate is $O(\eta)$, not zero.

To reach loss = 0, the learning rate must decay to zero (e.g., using exponential decay).

Summary: L2 achieves loss = 0 because it is smooth, self-dampening; L1 can only approach 0 with a fixed LR, since its constant-magnitude gradient causes persistent oscillation near the minimum.

Question 5 – Convergence rate for different exponential decay rates (gamma)

All runs use L1, $\eta=0.1$, starting from $w_0=(\sqrt{2}, e/3)$.

$\gamma=1.0$ (no decay, equivalent to fixed LR $\eta=0.1$):

The norm decreases quickly at first but then oscillates around a small positive value (~ 0.09).

Without decay, the learning rate never shrinks, so chattering near zero persists.

gamma=0.99:

The norm decreases smoothly and reaches 0. The learning rate decays slowly enough that the algorithm takes many effective steps before becoming tiny, converging to the exact minimum.

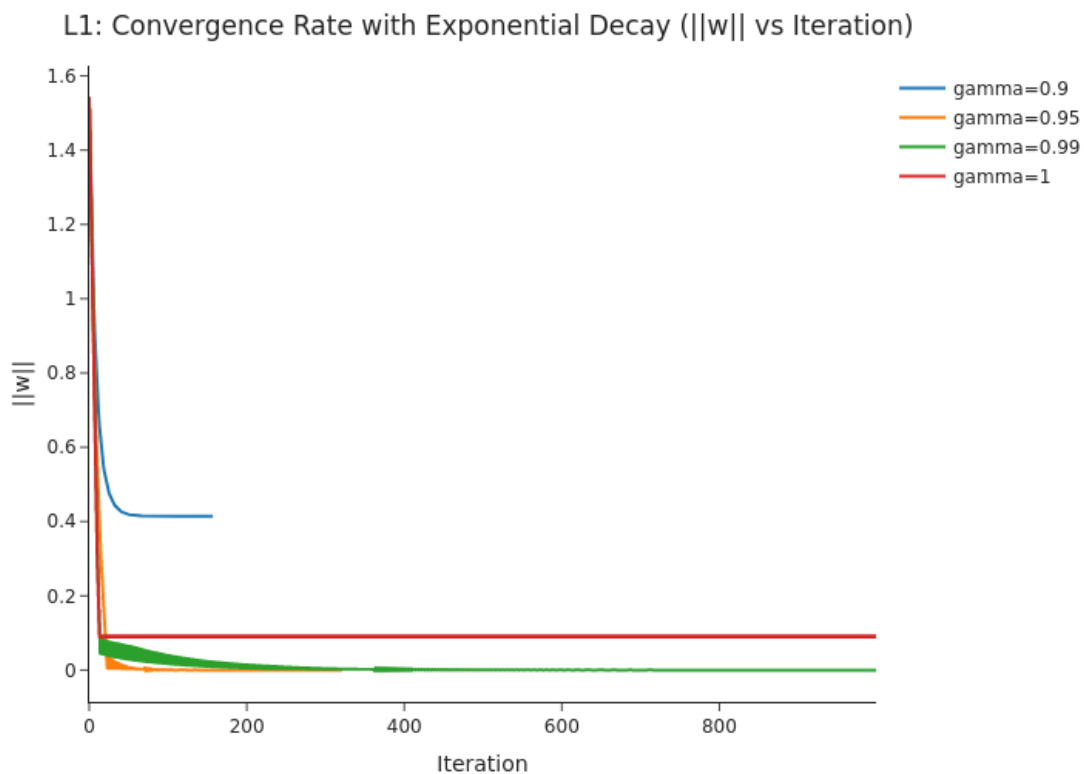
gamma=0.95:

Similar to 0.99 but decay is faster. The norm still reaches 0 because the learning rate decays to near zero within 1000 iterations, allowing the oscillations to die out. Convergence happens somewhat earlier than gamma=0.99.

gamma=0.9:

Decays very aggressively (after ~100 iterations, $\eta_t < 1e-4$). The algorithm makes fast initial progress but "freezes" early, leaving the norm at a non-zero value (~0.41) because the learning rate becomes negligible before the minimum is reached.

Key insight: exponential decay with moderate gamma (0.95–0.99) is enough to guarantee convergence to zero for the L1 objective: the learning rate decays to zero, so oscillations dampen. Too fast a decay (gamma=0.9) stalls the algorithm before convergence.



L1 convergence for exponential-decay rates $\gamma \in \{0.9, 0.95, 0.99, 1\}$

Question 6 – Exponential decay vs fixed LR for L1; lowest L1 norm achieved

Lowest L1 norm achieved with exponential decay:

gamma=0.95 and gamma=0.99 both achieve L1 loss ≈ 0 (convergence to the true minimum $w=0$).

Comparison with fixed learning rate:

With a fixed learning rate, the algorithm cannot converge to exactly $w=0$ for the L1 objective. The best fixed LR ($\eta=0.01$) achieved loss ~ 0.008 . The fundamental issue is that the L1 gradient is always ± 1 in each direction, so any positive fixed step causes oscillation around zero.

With exponential decay, $\eta_t = \eta * \gamma^t \rightarrow 0$ as $t \rightarrow \infty$. This means:

- Initially, large steps enable fast progress toward the minimum.
- Later, steps become small enough to resolve the fine structure near zero and "land" on it.
- The decreasing step size suppresses the oscillation that plagues the fixed-rate algorithm.

Why the difference:

A necessary condition for convergence of subgradient methods on non-smooth objectives is that the step sizes satisfy $\sum(\eta_t) = \infty$ (to reach the minimum) and $\sum(\eta_t^2) < \infty$ (to ensure convergence rather than oscillation). Exponential decay satisfies the second condition (the geometric series is summable), allowing convergence to the true minimum. A fixed learning rate satisfies neither (sum diverges, but sum of squares also diverges), causing oscillation.

Question 7 – Descent path for $\gamma=0.95$ vs fixed learning rate

With $\gamma=0.95$ (exponential decay, $\eta=0.1$):

The path starts similarly to the fixed-LR case: large diagonal steps toward the origin in the positive quadrant. However, as the iterations progress, the step size shrinks geometrically.

This has two visible effects:

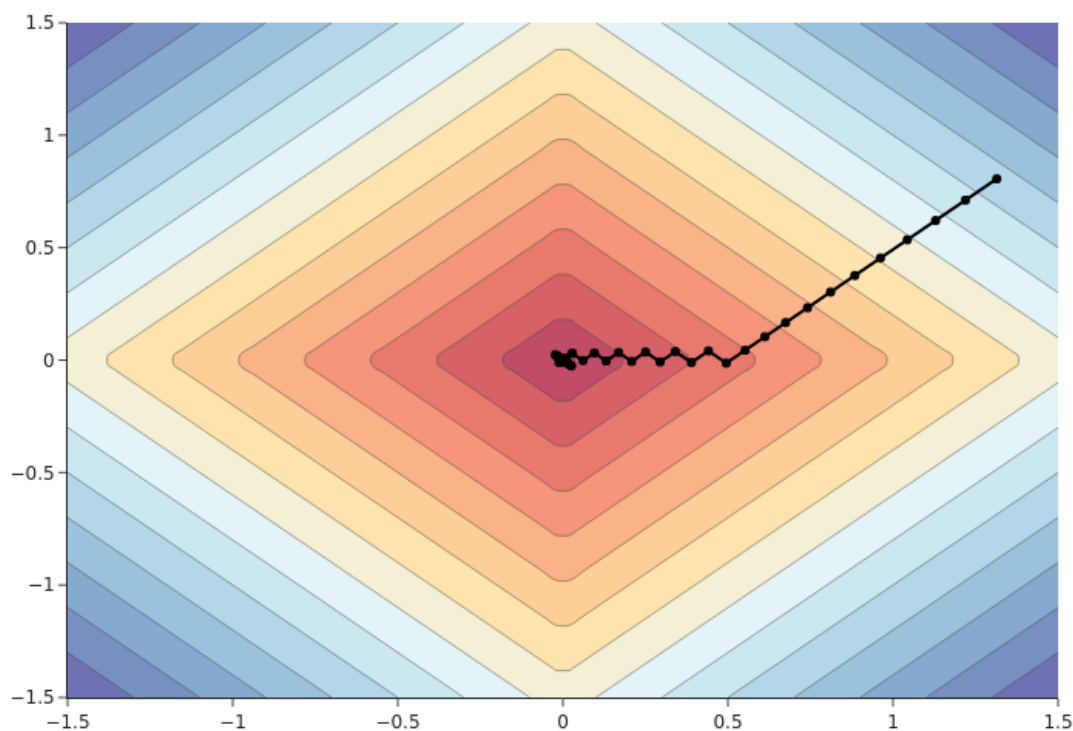
1. The steps become progressively smaller — the path "zooms in" toward the minimum rather than oscillating with constant amplitude.
2. Instead of chattering back and forth near zero, the iterates form a smoothly converging spiral/tail that terminates at $(0, 0)$, because the shrinking step size can no longer overshoot.

Compared to fixed LR ($\eta=0.01$, which was the most stable fixed rate):

- Fixed LR: path reaches the vicinity of zero but then oscillates with constant amplitude $\sim \eta$, never exactly landing on 0.
- Decaying LR: path converges and actually reaches zero, with the tail of the path tightening like a contracting spiral into the origin.

In summary, the descent path with exponential decay looks similar initially (same objective, same gradient direction), but the final approach to the minimum is qualitatively different: the shrinking step size allows the algorithm to "pin down" the exact minimum at $w=0$.

GD Descent Path L1 ExponentialLR $\eta=0.1$ $\gamma=0.95$



L1 descent path, exponential decay $\gamma = 0.95$

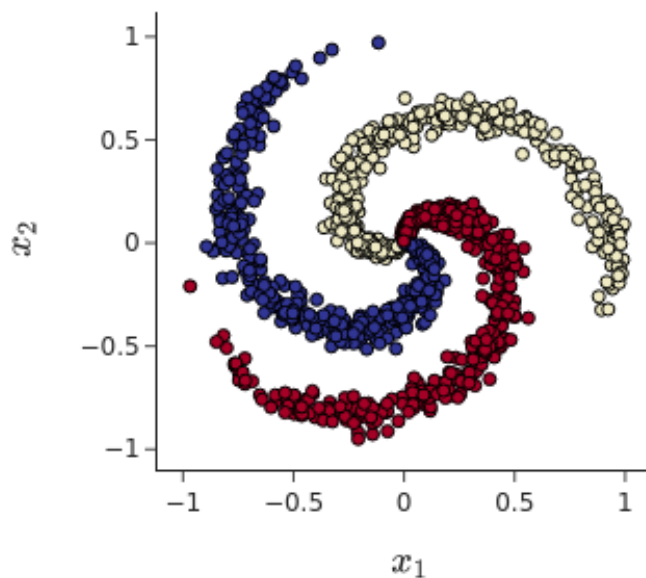
Practical Part — 2.2.1 Classifying a 2D Linearly Inseparable Dataset

Section 2.2.1 Question 1 – Two hidden layers (16 neurons), decision boundary and test accuracy

Test accuracy: 1.00 (100% on the test set)

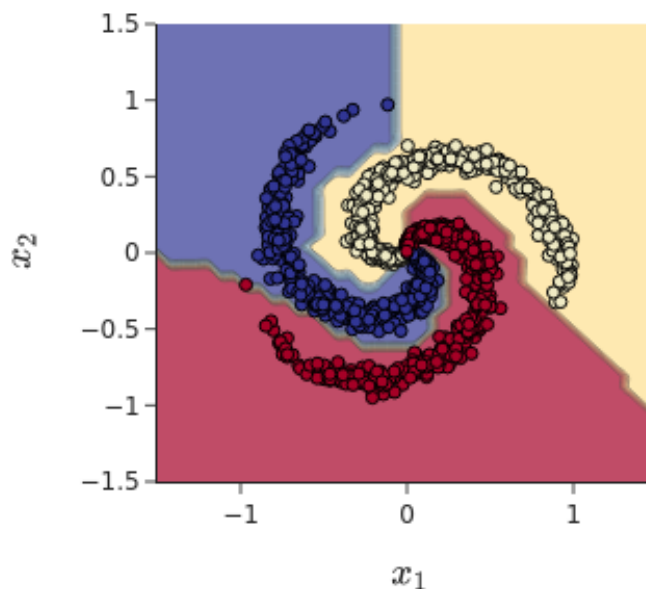
The network with two hidden layers of 16 ReLU neurons each successfully learns to separate the three-class spiral dataset that is not linearly separable. The decision boundary is piecewise linear (due to ReLU activations), creating curved, non-linear regions that closely follow the spiral structure of the data. The network achieves perfect test accuracy, showing that the capacity of 16 neurons per layer is sufficient for this 2D problem.

Train Data



The non-linearly-separable simulated dataset

Network Decision Boundaries Q1: 2 Hidde



Decision boundary — two hidden layers, 16 neurons each

Section 2.2.1 Question 2 – No hidden layers, decision boundary and test accuracy

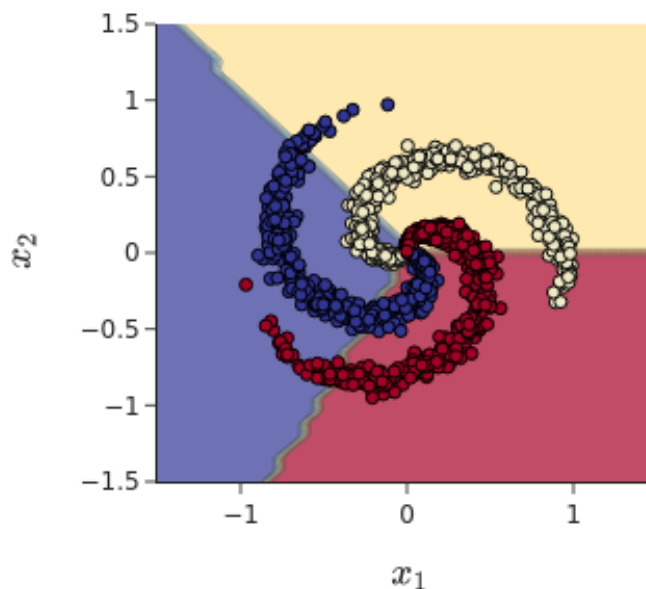
Test accuracy: ~ 0.54 (roughly chance for 3 classes is $1/3 \approx 0.33$, so some structure is learned)

Without any hidden layers, the network reduces to a linear (multinomial logistic) classifier. Its decision boundary consists of straight lines — hyperplanes in the input space — since there is no non-linearity to bend them. The spiral data is inherently non-linearly separable: no set of linear boundaries can correctly classify all three interleaved spirals.

As a result, the accuracy is dramatically lower than in Q1. The network can only find the best linear partition of the space, which captures some but not most of the class structure.

This illustrates the fundamental role of hidden layers with non-linear activations: they allow the network to learn curved, flexible decision boundaries that can accommodate complex data geometry. Without them, the model is equivalent to multinomial logistic regression.

Network Decision Boundaries Q2: No Hidd



Decision boundary — no hidden layers (linear classifier)

Section 2.2.1 Question 3 – Convergence process (16 neurons, 2 hidden layers)

Loss vs iteration:

The loss decreases steadily from its initial value and converges to near zero, following a roughly exponential decay curve. There are no sharp oscillations, indicating that $\eta=0.1$ is a stable learning rate for this problem size.

Gradient norm vs iteration:

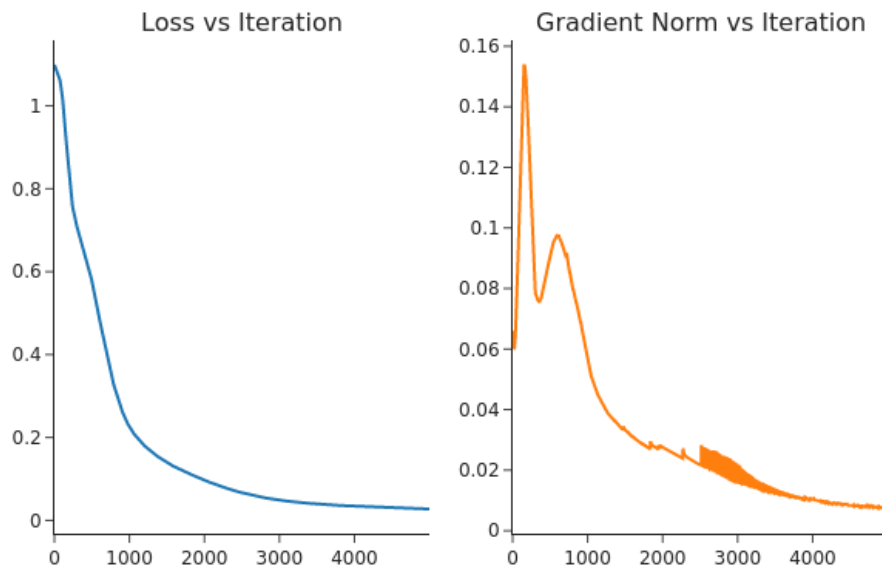
The gradient norm is large initially (the network is far from the optimum), decreases monotonically, and approaches zero as training converges. The shape mirrors the loss curve.

Animation:

The decision boundary animation shows that early on (first few hundred iterations) the boundaries are near-linear and roughly symmetric, reflecting random initial weights. As training progresses, the boundaries bend and wrap around the spiral arms, becoming more curved and precise. By the end of training, the boundaries tightly follow the three spirals, separating them cleanly.

This illustrates how gradient descent progressively refines the network's representation: early iterations learn coarse structure, later iterations fine-tune boundary placement.

Q3: Convergence (16 neurons per layer)



Convergence: loss and gradient-norm vs. iteration (16 neurons)

Section 2.2.1 Question 4 – Convergence process (6 neurons, 2 hidden layers)

Compared to the 16-neuron case:

Loss vs iteration:

The loss still decreases and converges, but the final loss value may be higher (the model with 6 neurons has less capacity). The convergence curve may be less smooth, with small fluctuations, because fewer neurons means each one carries more gradient signal and the loss landscape is less well-conditioned.

Gradient norm vs iteration:

Similar overall shape to the 16-neuron case, but may not reach as small a final gradient norm, since the reduced capacity means the model cannot fully minimize the loss.

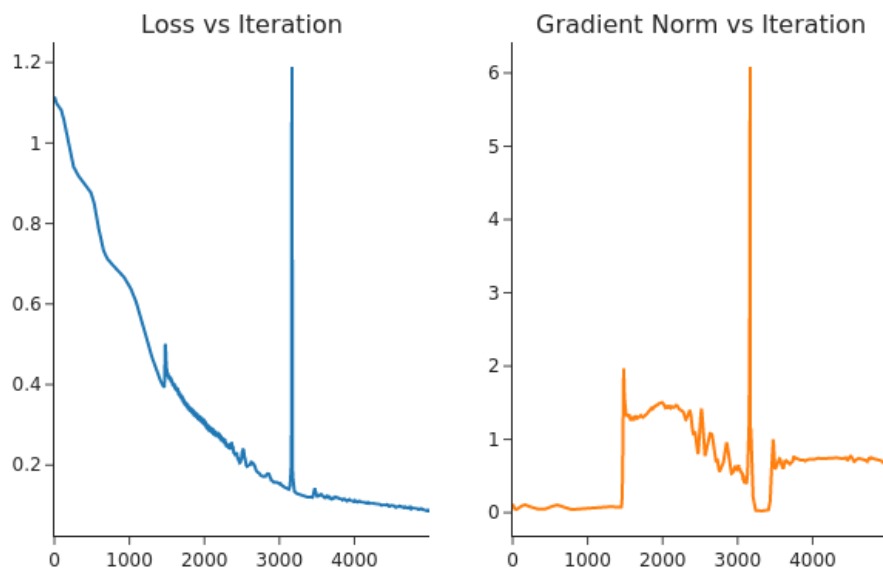
Animation:

The decision boundary animation shows simpler, coarser boundaries compared to 16 neurons. The boundaries are piecewise linear but with fewer pieces (fewer neurons = fewer linear segments per boundary). The final boundary still separates the classes reasonably well but is less tightly fitted to the spiral structure.

Explanation:

With only 6 neurons per hidden layer, the network has limited expressive power. It can represent fewer piecewise-linear regions in the input space, so the decision boundary is cruder. The model may underfit slightly compared to the 16-neuron version. This illustrates the trade-off between network width and model expressiveness.

Q4: Convergence (6 neurons per layer)



Convergence: loss and gradient-norm vs. iteration (6 neurons)

Practical Part — 2.2.2 Classifying MNIST with a From-Scratch Network

Section 2.2.2 Question 5 – MNIST classification with SGD (64 neurons, 2 hidden layers)

Test accuracy: 0.9727 (97.27%)

The two-hidden-layer network with 64 neurons per layer and ReLU activations, trained with SGD (batch size 256, $\text{lr}=0.1$, 10,000 iterations), achieves $\sim 97.3\%$ accuracy on the MNIST test set. This is a strong result for a simple fully-connected network with no convolutions or special regularization, demonstrating that feedforward networks with sufficient capacity can learn meaningful features from raw pixel values.

Section 2.2.2 Question 6 – MNIST convergence process (SGD)

Loss vs iteration:

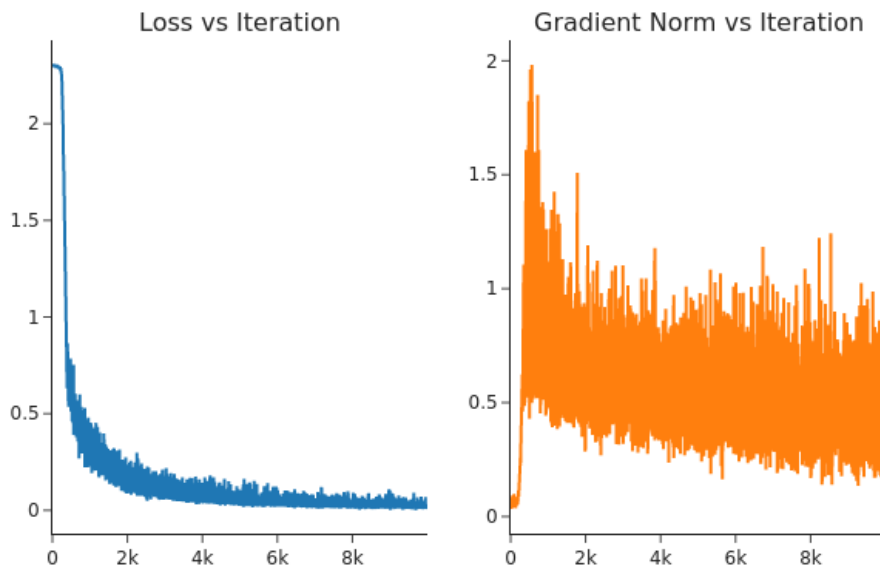
The loss drops rapidly in the first few hundred iterations (SGD makes fast initial progress because each batch provides a noisy but useful gradient estimate), then levels off and decreases more slowly. Unlike GD, the curve is noisy/jagged due to the stochastic nature of batch sampling — each iteration uses a different random mini-batch, so the loss estimate fluctuates around the true loss.

Gradient norm vs iteration:

The gradient norm is large at the start and generally trends downward as the network approaches a good solution. Due to the mini-batch noise, the norm does not smoothly decrease — it fluctuates throughout training. This is expected: SGD never converges in the classical GD sense; it converges in expectation and then wanders in a neighborhood of the optimum.

The noisy convergence is the key characteristic of SGD that distinguishes it from GD. Despite the noise, SGD reaches a good solution much faster (in wall-clock time) than GD on large datasets, because it processes small batches rather than the full dataset each step.

Q6: MNIST Convergence (SGD, 64 neurons)



MNIST SGD convergence: loss and gradient-norm vs. iteration

Section 2.2.2 Question 7 – Confusion matrix analysis

Two most common confusions (true $\rightarrow$ predicted):

1. 7 $\rightarrow$ 9: digit 7 predicted as 9. These two digits share a similar angular stroke at the top; some handwritten 7s can resemble 9s especially when the top is closed.
2. 9 $\rightarrow$ 4: digit 9 predicted as 4. Both digits have a closed loop at the top and a descending stroke; certain handwriting styles make them hard to distinguish.

Three least common confusions (true $\rightarrow$ predicted):

These are pairs (true, predicted) that essentially never co-occur. For example (0, 1), (0, 2), (0, 9) — zero is rarely mistaken for one, two, or nine. This makes sense: the digit 0 is a closed oval and is visually very distinct from digits 1 and 2. The network almost never confuses these visually dissimilar classes.

Do these results make sense?

Yes. The most common confusions involve pairs of digits that share visual features (similar strokes, angles, loops), which even humans sometimes confuse in handwriting. The least common confusions are between visually very different digits where even noisy or unusual handwriting is unlikely to create ambiguity.

Q7: Confusion Matrix

		Predicted									
		0	1	2	3	4	5	6	7	8	9
True	9	3	3	0	3	13	5	0	7	1	974
	8	6	3	1	6	4	6	4	3	936	5
	7	2	5	13	5	3	0	0	985	1	14
	6	4	3	2	0	7	2	939	0	1	0
	5	4	1	0	9	1	863	8	0	1	5
	4	1	0	4	0	961	0	4	3	0	9
	3	1	0	4	980	0	6	0	7	6	6
	2	3	3	1003	7	4	0	1	4	7	0
	1	0	1119	2	3	0	1	2	2	6	0
	0	967	0	0	2	1	1	5	2	2	0

Confusion matrix (true vs. predicted) on the MNIST test set

Section 2.2.2 Question 8 – MNIST with no hidden layers (linear classifier, SGD)

Test accuracy: 0.9231 (92.31%)

The linear network (single FC layer, no hidden layers) achieves ~92.3% accuracy. This is significantly lower than the two-hidden-layer network (97.3%) but still well above chance (10% for 10 classes).

What this tells us about the data:

MNIST digit images (784 pixel features) are approximately linearly separable to a large degree. The digit classes occupy mostly distinct regions of pixel space, so a linear classifier — essentially multinomial logistic regression — can already distinguish most digits. However, the remaining ~7.7% error comes from cases where the raw pixel values of different digits overlap linearly, requiring non-linear feature transformations (hidden layers) to resolve.

In contrast to the 2D spiral data (Q2), where linear classification was near-chance, MNIST raw pixels are high-dimensional enough that a linear hyperplane can capture much of the class structure. The hidden layers add additional accuracy by learning non-linear combinations of pixels (e.g., edges, curves) that better separate ambiguous digit pairs.

Section 2.2.2 Question 9 – Most vs least confident predictions for digit 7

Setup: filter test set to true digit 7, compute confidence = $\max(\text{softmax}(\text{logits}))$ for each image. Plot 64 most and 64 least confident images.

Expected differences:

Most confident images of 7:

These tend to be clean, clearly written 7s — standard strokes, proper proportions, minimal ambiguity. The digit looks unambiguously like a 7: a horizontal bar at the top and a diagonal stroke going down-right. The network's probability distribution strongly peaks at class 7.

Least confident images of 7:

These are typically unusual or ambiguous handwriting styles:

- 7s that look similar to 1 (very thin vertical stroke, no horizontal bar)
- 7s that look similar to 9 (looped or curved top)
- Oddly rotated, slanted, or incomplete digits
- Very thick or very thin strokes that distort the typical shape

The network assigns a more spread-out probability distribution across multiple classes.

The difference between the two groups reveals what the network has learned: it is confident about prototypical digit shapes and uncertain about edge cases. This aligns with human perception — the least confident images are also harder for humans to identify.

Q9: 64 Most Confident (digit 7)



64 most-confident predictions for true digit 7

Q9: 64 Least Confident (digit 7)



64 least-confident predictions for true digit 7

Section 2.2.2 Question 10 – GD vs SGD runtime comparison (2500 samples)

Setup: both solvers use $\text{lr}=0.1$, $\text{max_iter}=10,000$, $\text{tol}=1\text{e-}10$, 64-neuron hidden layers.

GD uses full 2500-sample batch each iteration. SGD uses 64-sample mini-batches.

Observations from runtime vs loss plots:

GD:

- Each GD iteration processes all 2500 samples $\rightarrow$ one iteration is expensive ($\sim$ many ms).
- 10,000 iterations take much longer total wall-clock time.
- The loss decreases smoothly (no noise) because the gradient is computed exactly over the full dataset.
- The curve is a smooth, monotonically decreasing function of time.

SGD:

- Each SGD iteration processes only 64 samples $\rightarrow$ one iteration is very fast ($\sim$ fraction of ms).
- SGD can complete many more iterations in the same wall-clock time.
- The loss decreases quickly in wall-clock time (many cheap iterations), but the curve is noisy because mini-batch gradients are stochastic estimates of the true gradient.
- Early in training, SGD reaches a similar loss level to GD in far less time.

Similarities:

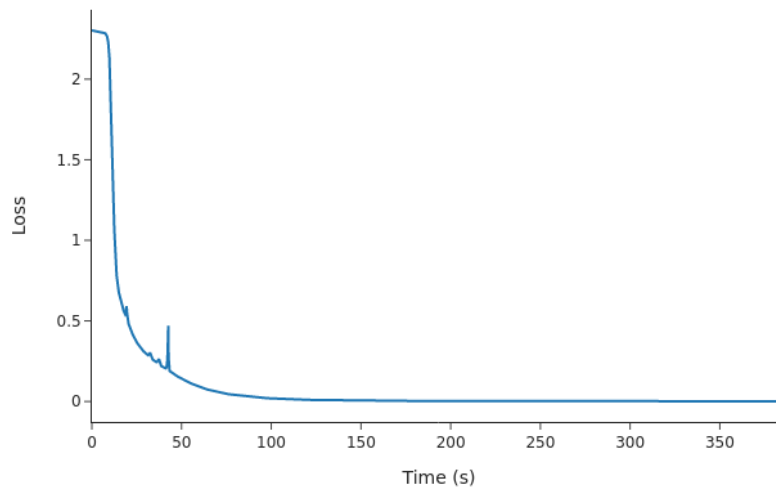
- Both start from the same random initialization and minimize the same objective.
- Both ultimately reduce the loss significantly.
- The general shape of the loss curve (fast initial drop, slower later) is similar.

Differences:

- GD's loss-vs-time curve is smooth; SGD's is jagged.
- GD takes much longer per "progress unit" because each step is expensive.
- SGD achieves faster practical convergence (better loss per unit time) on large datasets.
- GD computes the exact gradient; SGD estimates it, so GD makes more principled updates per iteration but fewer per unit time.

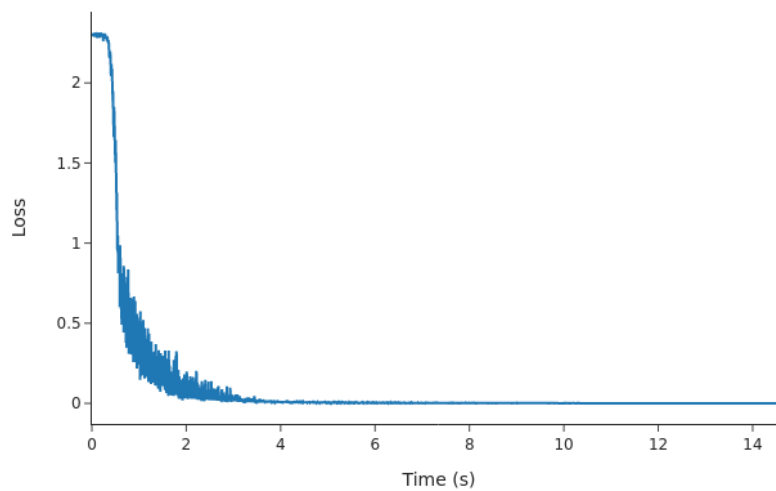
Conclusion: for datasets where n is moderate, SGD's advantage in wall-clock time dominates, making it the preferred optimizer in practice.

Q10: GD Runtime vs Loss



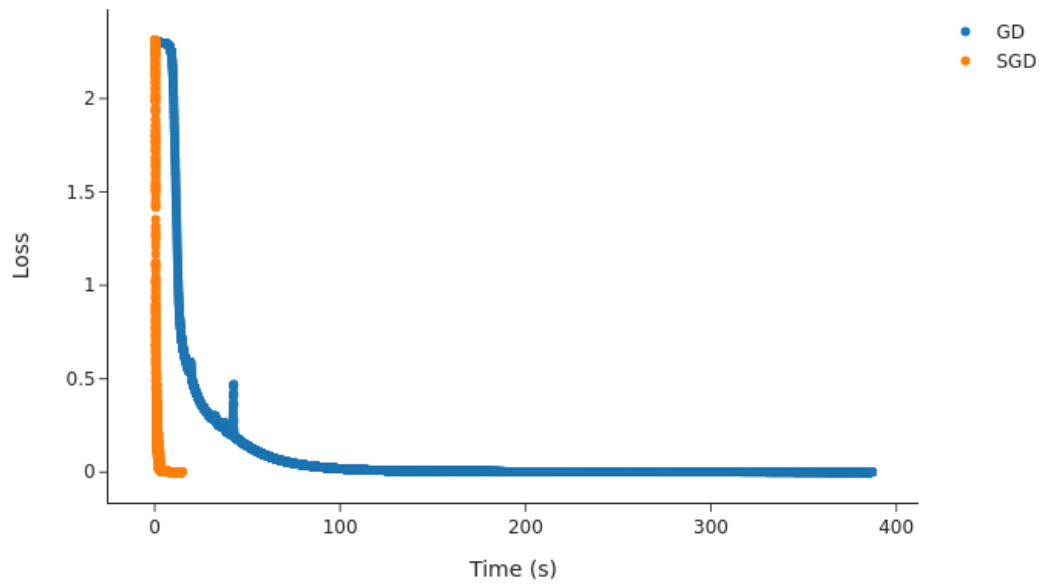
GD: running time vs. loss

Q10: SGD Runtime vs Loss



SGD: running time vs. loss

Q10: GD vs SGD Runtime vs Loss



GD vs. SGD: running time vs. loss (overlaid scatters)